

类的静态成员

北京理工大学计算机学院
金旭亮



定义类的静态字段

```
class Employee {  
    String name;  
    long salary;  
    short employee_id;  
    static int total_employees;  
    ...  
}
```

类静态字段的访问方法



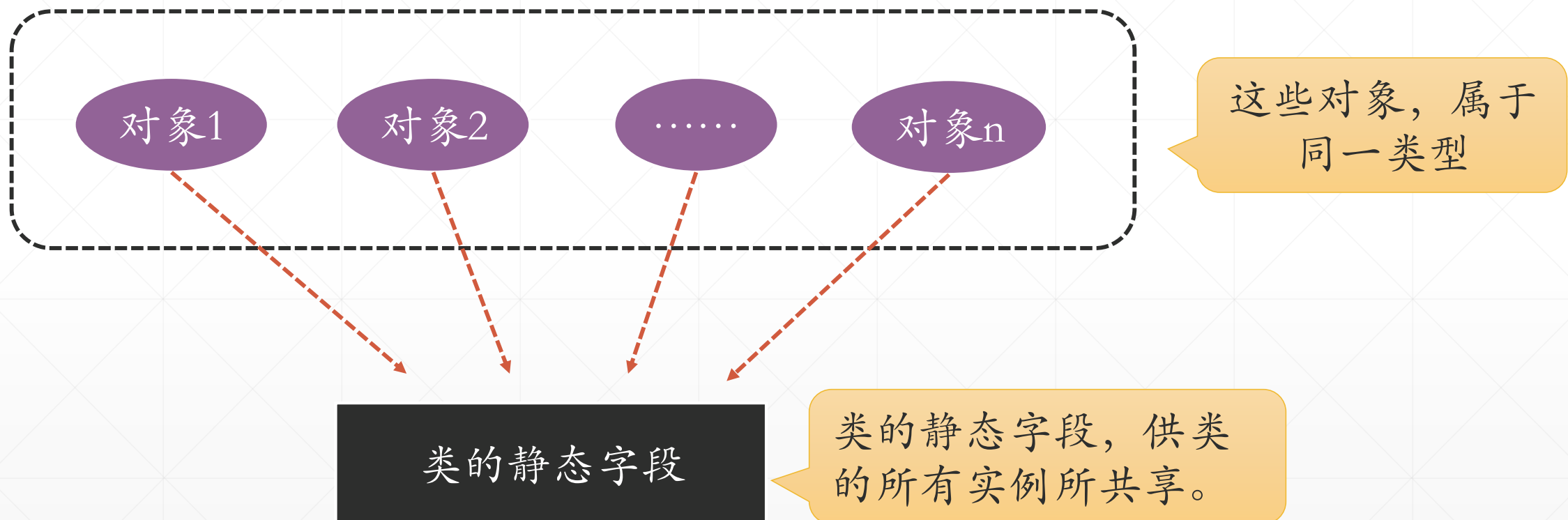
可以用对象变量或类名作为前缀访问静态数据：

1. `Employee newhire = new Employee();`
2. `newhire.total_employees=1;`
3. `Employee.total_employees=1;`



实际开发中，推荐使用类名而不是对象变量来访问静态字段（或调用静态方法）。

类静态字段的重要性质



类的静态初始化块



可以使用“**静态初始化块**”初始化类的静态字段：

```
public class Test {  
    public static int value;
```

```
    static {  
        value = 100;  
    }
```

```
}
```

类的“**静态**”初始化块



请运行TestStaticInitializeBlock.java示例，观察输出结果，总结出“静态初始化块的执行顺序”（参考解答在下页PPT）

静态初始化块的执行顺序



静态初始化块只执行一次。



创建子类型的对象时，也会导致父类型的静态初始化块的执行。

类的静态方法

```
class Employee
{
    String name;
    long salary;
    short employee_id;
    static int total_employees;
    static void clear(){
        total_employees = 0;
    }
}
```



类的静态方法只能访问类的静态成员！

一个有趣的问题：



静态方法中只允许访问静态数据，那么，能不能静态方法中访问类的实例成员（即那些没有附加`static`关键字的字段或方法）？

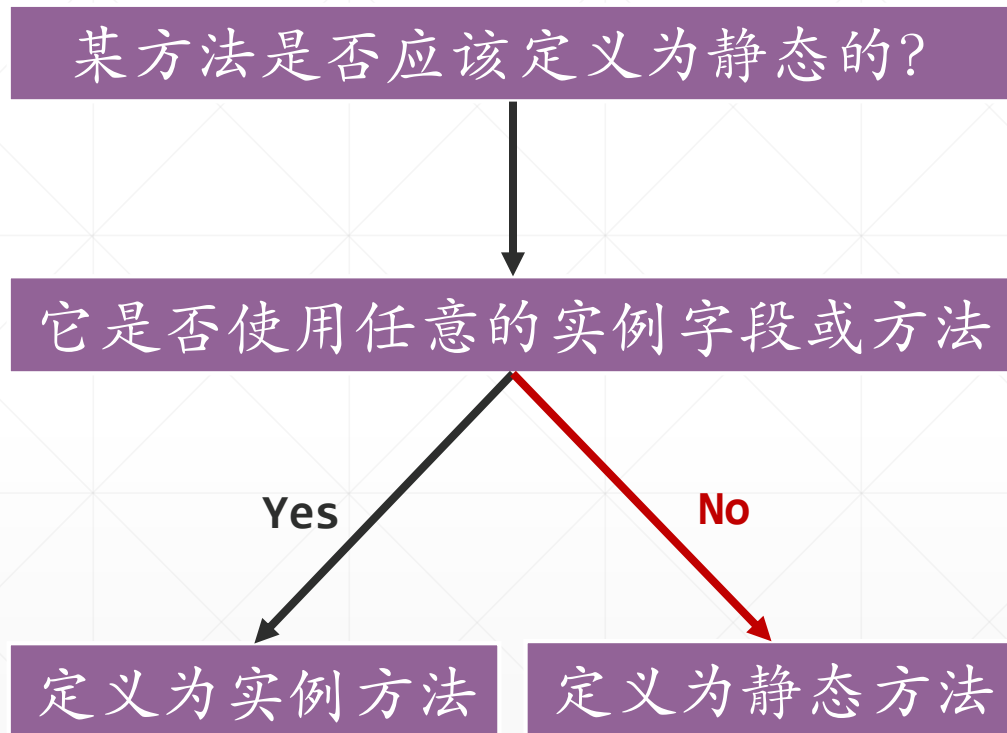
请编写代码验证你的想法。

（答案参看后页PPT）

参考解答:

```
public class StaticVisitInstance {  
  
    private int value = 100;  
  
    private void increaseValue() {  
        value++;  
    }  
    //只需要给静态方法传入一个对象引用,就能通过它访问到类的实例成员  
    public static void staticMethodDemo(StaticVisitInstance obj) {  
        obj.value = 1000;  
        obj.increaseValue();  
        System.out.println(obj.value);//1001  
    }  
  
    public static void main(String[] args) {  
        staticMethodDemo(new StaticVisitInstance());  
    }  
}
```

使用静态还是实例方法？



在定义时没有加“static”的方法或字段，是类的“实例(instance)”成员。

课后实践



使用类的静态字段和构造函数，我们可以跟踪某个类所创建对象的个数。请写一个类，在任何时候都可以向它查询“你已经创建了多少个对象？”

想一想：你写的这个类能在多线程环境中工作正常吗？应该怎样编写一个能在多线程环境中正常工作的类？（可以在学完“多线程”内容后再做这个练习）

枚举类型



枚举类型

- 定义:

```
enum Size{ SMALL , MEDIUM , LARGE }
```

- 使用:

```
Size s = Size.SMALL;
```

```
//从字符串转换为枚举
```

```
Size t = Size.valueOf("SMALL") ;
```



枚举类型是JDK 5.0引入的。

枚举值的foreach迭代

```
public class UseMyEnum {  
    public static void main(String[] args) {  
        //输出: ONE,TWO,THREE,  
        for (MyEnum value : MyEnum.values()) {  
            System.out.print(value + ",");  
        }  
    }  
}  
  
enum MyEnum {  
    ONE, TWO, THREE  
}
```

示例：UseMyEnum.java



注意：枚举可用于switch语句中



动手动脑

- 仔细阅读示例: EnumTest.java, 运行它, 分析运行结果
- 你能得到什么结论? 你掌握了枚举类型的基本用法了吗?

```
public class EnumTest {  
  
    public static void main(String[] args) {  
        Size s = Size.SMALL;  
        Size t = Size.LARGE;  
        //s和t引用同一个对象?  
        System.out.println(s == t);    //false  
        //s和t的内容相等?  
        System.out.println(s.equals(t)); //false  
        //是原始数据类型吗?  
        System.out.println(s.getClass().isPrimitive()); //false  
        //从字符串中转换  
        Size u = Size.valueOf("SMALL");  
        System.out.println(s == u);    //true  
        //列出它的所有值: SMALL, MEDIUM, LARGE,  
        for (Size value : Size.values()) {  
            System.out.print(value+",");  
        }  
    }  
}  
  
enum Size {SMALL, MEDIUM, LARGE};
```

牢记一



枚举类型是引用类型！



枚举不属于原始数据类型，它的每个具体值都引用一个特定的对象，相同的值则引用同一个对象。



可以使用“==”和“equals()”方法直接比对枚举变量的值，换句话说，对于枚举类型的变量，“==”和“equals()”方法执行的结果是等价的。

动手动脑



Java编程语言中的enum类型，实际上派生自 `java.lang.Enum<E>` 类型。



在IntelliJ中输入Enum这个单词，压住Ctrl键，然后用鼠标点击，就能打开Enum<E>类的源代码，请你打开这个类的源代码，尝试着仔细读读，看看能读懂多少？